

Objetivo

Diagnosticar una falla de resolución DNS desde un Pod sin modificar CoreDNS. Diferenciar un problema del cliente de un problema del servicio DNS del clúster. Interpretar dnsPolicy y /etc/resolv.conf dentro de un Pod. Restaurar la resolución mediante ClusterFirst y validar nombres corto y FQDN.

Requisitos previos

- Haber completado la Práctica 6 y la Práctica complementaria 6.1.
- Conservar el namespace lab6 y los Services backend-svc y frontend-svc.
- Tener CoreDNS operativo en Minikube.
- Tener Docker Desktop y kubectl disponibles.
- Disponer del archivo dns-internal-scenario6-2.yaml proporcionado con esta práctica.

Duración estimada

15 minutos

Introducción

En este reto un Pod de diagnóstico dentro de lab6 no puede resolver backend-svc, aunque el Service existe y otros componentes del clúster permanecen saludables. No conocerás inicialmente la causa. Deberás diferenciar una falla del cliente de una falla de CoreDNS, inspeccionar la configuración DNS efectiva y aplicar la corrección mínima sin modificar los componentes del sistema.

Instrucciones



Escenario del reto

El Service `backend-svc` funciona después de completar la complementaria 6.1. Sin embargo, un nuevo Pod de diagnóstico dentro de `lab6` no puede resolver su nombre DNS.

Debes determinar:

1. ¿backend-svc existe y conserva su ClusterIP?
2. ¿CoreDNS está operativo?
3. ¿La falla ocurre en todo el clúster o solamente en dns-client?
4. ¿Qué configuración DNS utiliza el Pod?
5. ¿Cuál es la corrección mínima sin modificar CoreDNS?

 Copiar

Distribución sugerida

Preparación	2 min
Reto 1	4 min
Reto 2	5 min
Reto 3	4 min
Total	15 min

 Copiar

Preparación del escenario

- **Paso 1.** Comprueba que `backend-svc` existe y conserva destinos antes de iniciar un diagnóstico DNS.

```
kubectl get service backend-svc \
-n lab6

kubectl get endpoints backend-svc \
-n lab6
```

 Copiar

Salida esperada aproximada:

NAME	TYPE	CLUSTER-IP	PORT(S)
backend-svc	ClusterIP	10.x.x.x	80/TCP

NAME	ENDPOINTS
backend-svc	10.244.x.x:80,10.244.x.x:80

 Copiar

- **Paso 2.** Confirma que CoreDNS está `Running`, estableciendo una línea base antes de crear el cliente defectuoso.

```
kubectl get pods \  
-n kube-system \  
-l k8s-app=kube-dns
```

 Copiar

Salida esperada aproximada:

NAME	READY	STATUS
coredns-<hash>-<id>	1/1	Running

 Copiar

Aplicar el estado inicial del reto

- **Paso 3.** Descarga `dns-internal-scenario6-2.yaml`.

```
curl -L \  
-o dns-internal-scenario6-2.yaml \  
https://raw.githubusercontent.com/Netec-  
Mx/CKA/refs/heads/main/labs/lab9/dns-internal-  
scenario6-2.yaml
```

 Copiar

- **Paso 4.** Aplica `dns-internal-scenario6-2.yaml` sin inspeccionarlo previamente para crear el Pod que reproduce el incidente.

```
kubectl apply -f dns-internal-scenario6-2.yaml
```

 Copiar

Salida esperada:

```
pod/dns-client created
```

 Copiar

- **Paso 5.** Espera hasta que el Pod esté listo antes de interpretar las pruebas de resolución.

```
kubectl wait \  
--for=condition=Ready \  
pod/dns-client \  
-n lab6 \  
--timeout=60s
```

 Copiar

Salida esperada:

```
pod/dns-client condition met
```

 Copiar

IMPORTANTE: No abras el manifiesto del escenario antes de completar los Retos 1 y 2. No modifiques CoreDNS durante esta práctica.

Reto 1. Determinar si el fallo es global o del cliente

Tiempo sugerido: 4 minutos

Reto 1.1. Reproducir la falla

- **Paso 6.** Intenta resolver el nombre corto `backend-svc` desde `dns-client` y registra el resultado.

```
kubectl exec \  
-n lab6 \  
dns-client -- \  
nslookup backend-svc
```

 Copiar**Salida esperada durante el incidente:**

La consulta debe fallar o terminar por timeout porque el Pod no utiliza correctamente el DNS del clúster.

Reto 1.2. Comprobar otro cliente saludable

- **Paso 7.** Utiliza un Pod frontend existente para comprobar que el mismo nombre puede resolverse desde otro cliente del namespace.

```
FRONTEND_POD=$(kubectl get pod \
-n lab6 \
-l app=frontend \
-o jsonpath='{.items[0].metadata.name}')

kubectl exec \
-n lab6 \
"$FRONTEND_POD" -- \
sh -c 'getent hosts backend-svc 2>/dev/null ||
nslookup backend-svc'
```

 Copiar

Salida esperada aproximada:

```
10.x.x.x backend-svc.lab6.svc.cluster.local
```

 Copiar

La salida exacta depende de las herramientas disponibles, pero debe resolver la ClusterIP de `backend-svc`.

Evidencia requerida

```
[ ] backend-svc existe.
[ ] backend-svc tiene endpoints.
[ ] CoreDNS está Running.
[ ] Otro Pod puede resolver backend-svc.
[ ] dns-client es el único cliente con falla.
```

 Copiar

Reto finalizado

Resultado esperado: Falla DNS reproducida y aislada correctamente en `dns-client`, comprobando que `backend-svc`, sus endpoints y CoreDNS permanecen operativos.



Reto 2. Inspeccionar la configuración DNS del Pod

Tiempo sugerido: 5 minutos

Reto 2.1. Consultar resolv.conf

- **Paso 8.** Revisa `/etc/resolv.conf` dentro de `dns-client` utilizando `sh -c` para impedir que Git Bash transforme la ruta Linux en una ruta de Windows.

```
kubectl exec \  
-n lab6 \  
dns-client -- \  
sh -c 'cat /etc/resolv.conf'
```

[Copiar](#)

Evidencia esperada durante el incidente

La configuración no debe mostrar el servidor DNS y search domains normales de Kubernetes; observarás una configuración distinta a la utilizada por los Pods saludables.

Reto 2.2. Revisar dnsPolicy

- **Paso 9.** Consulta la política DNS efectiva del Pod para determinar si Kubernetes está aplicando la configuración estándar del clúster.

```
kubectl get pod dns-client \  
-n lab6 \  
-o jsonpath='{.spec.dnsPolicy}{"\n"}'
```

[Copiar](#)

- **Paso 10.** Consulta también una posible configuración DNS explícita para identificar si el Pod sobrescribe el resolver proporcionado por Kubernetes.

```
kubectl get pod dns-client \  
-n lab6 \  
-o jsonpath='{.spec.dnsConfig}{"\n"}'
```

[Copiar](#)

Reto 2.3. Formular la causa

Debes poder explicar:

CoreDNS está _____.

 Copiar

Otros Pods _____ backend-svc.

dns-client utiliza dnsPolicy _____.

Su configuración DNS apunta a _____.

Por tanto, la falla está en _____ y no en CoreDNS.

Reto finalizado

Resultado esperado: Configuración DNS del Pod diagnosticada mediante `dnsPolicy`, `dnsConfig` y `/etc/resolv.conf`, identificando por qué el cliente no utiliza correctamente el DNS del clúster.

Reto 3. Restaurar DNS con la política correcta

Tiempo sugerido: 4 minutos

Restricciones

- [] No modifiques CoreDNS.
- [] No modifiques kube-dns.
- [] No modifiques backend-svc.
- [] No cambies los Pods frontend/backend.
- [] Corrige únicamente dns-client.

 Copiar

Reto 3.1. Corregir el manifiesto

- **Paso 11.** Abre `dns-internal-scenario6-2.yaml` después de completar el diagnóstico y elimina la configuración DNS personalizada que impide usar el resolver del clúster.

La configuración final debe utilizar:

```
dnsPolicy: ClusterFirst
```

[Copiar](#)

Reto 3.2. Recrear el Pod

- **Paso 12.** Elimina únicamente `dns-client` y vuelve a aplicar el manifiesto corregido porque la política DNS forma parte de la especificación del Pod.

```
kubectl delete pod dns-client \
-n lab6

kubectl apply -f dns-internal-scenario6-2.yaml

kubectl wait \
  --for=condition=Ready \
  pod/dns-client \
  -n lab6 \
  --timeout=60s
```

[Copiar](#)

Salida esperada:

```
pod "dns-client" deleted
pod/dns-client created
pod/dns-client condition met
```

[Copiar](#)

Reto 3.3. Validar nombre corto y FQDN

- **Paso 13.** Comprueba que el nombre corto ahora resuelve mediante el search domain del namespace `lab6`.

```
kubectl exec \  
-n lab6 \  
dns-client -- \  
nslookup backend-svc
```

 Copiar

- **Paso 14.** Valida también el FQDN completo para confirmar la identidad DNS absoluta del Service.

```
kubectl exec \  
-n lab6 \  
dns-client -- \  
nslookup backend-svc.lab6.svc.cluster.local
```

 Copiar

Salida esperada aproximada:

```
Name:      backend-svc.lab6.svc.cluster.local  
Address: 10.x.x.x
```

 Copiar

- **Paso 15.** Confirma conectividad HTTP para demostrar que la resolución obtenida corresponde con un Service funcional.

```
kubectl exec \  
-n lab6 \  
dns-client -- \  
sh -c 'wget -qO- http://backend-svc | grep  
"BACKEND API"'
```

 Copiar

Salida esperada aproximada:

```
<html><body><h1>BACKEND API</h1><p>Pod: backend-...  
</p></body></html>
```

 Copiar

Reto finalizado

Resultado esperado: Política `ClusterFirst` restaurada y resolución validada mediante el FQDN de `backend-svc`, comprobando además

conectividad HTTP mediante el nombre corto dentro del namespace.

✓ Validación final

```
echo "=== Validacion complementaria 6.2 ==="

echo "dnsPolicy:"
kubectl get pod dns-client \
  -n lab6 \
  -o jsonpath='{.spec.dnsPolicy}{"\n"}'

echo ""
echo "Resolucion FQDN:"
kubectl exec \
  -n lab6 \
  dns-client -- \
  nslookup backend-svc.lab6.svc.cluster.local

echo ""
echo "HTTP:"
kubectl exec \
  -n lab6 \
  dns-client -- \
  sh -c 'wget -qO- http://backend-svc | grep "BACKEND API"'

echo "=== Fin de validacion ==="
```

[Copiar](#)

Salida esperada aproximada:

```
=== Validacion complementaria 6.2 ===
```

[Copiar](#)

```
dnsPolicy:
```

```
ClusterFirst
```

```
Resolucion FQDN:
```

```
Name: backend-svc.lab6.svc.cluster.local
```

```
Address: 10.x.x.x
```

```
HTTP:
```

```
<html><body><h1>BACKEND API</h1>...
```

```
=== Fin de validacion ===
```

- **Paso 16.** Elimina únicamente el Pod temporal después de conservar las evidencias.

```
kubectl delete pod dns-client \
-n lab6
```

[Copiar](#)

IMPORTANTE: Conserva `lab6`, sus Deployments, Services y el addon de Ingress. Solo debe eliminarse el Pod temporal `dns-client`.

Resolución de problemas

Problema 1. dns-client no inicia

```
kubectl get pod dns-client -n lab6
kubectl describe pod dns-client -n lab6
```

[Copiar](#)

Revisa scheduling e imagen antes de investigar DNS.

Problema 2. También falla DNS desde frontend

Comprueba CoreDNS y el Service DNS:

```
kubectl get pods \  
-n kube-system \  
-l k8s-app=kube-dns  
  
kubectl get service kube-dns \  
-n kube-system
```

 Copiar

Si otros Pods tampoco resuelven, el problema ya no corresponde al escenario previsto.

Problema 3. Git Bash transforma /etc/resolv.conf

Utiliza siempre:

```
kubectl exec -n lab6 dns-client -- \  
sh -c 'cat /etc/resolv.conf'
```

 Copiar

No ejecutes la ruta Linux directamente después de `--` en Git Bash.

Problema 4. DNS resuelve pero HTTP falla

```
kubectl get endpoints backend-svc -n lab6  
kubectl get pods -n lab6 -l app=backend
```

 Copiar

Si DNS devuelve la ClusterIP correcta, investiga Service, endpoints o Pods y no CoreDNS.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar el reto habrás diagnosticado una política DNS incorrecta dentro de un Pod, restaurado la configuración ClusterFirst y validado la resolución de backend-svc mediante nombre corto y FQDN.

Notas y consideraciones

- Esta práctica dura 15 minutos y reutiliza los Services creados en el Lab 10.
- No modifiques Deployments, Services, CoreDNS ni el ConfigMap de CoreDNS.

- En Git Bash, las rutas Linux dentro del contenedor se consultan mediante `sh -c` para evitar conversión automática de rutas.
- El Pod `dns-client` es temporal y debe eliminarse al finalizar.
- Esta práctica complementaria no incluye prompts de apoyo con IA.

URL de referencia

- [DNS for Services and Pods](#)
- [Debugging DNS Resolution](#)
- [Pod DNS Config](#)